

In-class live session

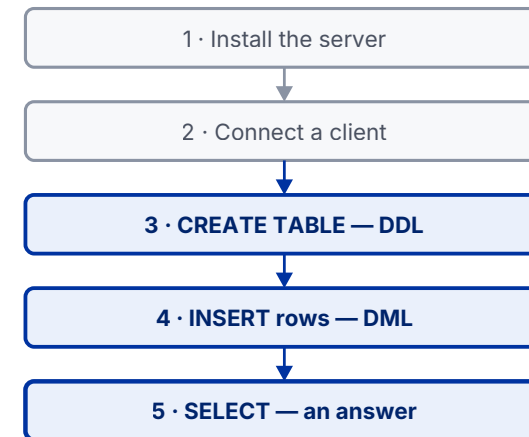
SQL, live

Today we install MySQL, build the UNIVERSITY database from an empty server, and run queries against it — on screen, in real time, mistakes included.

Everything you have seen so far has been on paper: diagrams, relations, algebra. By the end of this session there is a real server on a real machine answering real questions. HW2 asks you to do exactly this on your own, and it is due Wednesday.

Follow along if you can

If you have a laptop, install alongside me. If you would rather watch, watch — every command is in these slides and in the companion script, and nothing today is lost by watching first.



Two setup steps, then three SQL steps. Only the last three are the course; the first two are the toll you pay once.

Introduction

The plan, and the clock

Five things happen today, in this order. The install is the only part that can go badly, so it goes first and it gets a hard stop.

- 1:30** Why a live server, and what we are building
- 1:35** **Install MySQL** — hard stop at 1:55
- 1:55** Connect; CREATE DATABASE
- 2:05** **DDL** — four tables, keys, foreign keys
- 2:20** **DML** — insert rows; break the constraints on purpose
- 2:30** **SELECT-FROM-WHERE** — the exercise ladder
- 2:45** Close

The hard stop is real

If your install is not finished at 1:55, stop installing and start watching. A half-finished install is not worth the rest of the period, and office hours exist. You will not fall behind: the companion script rebuilds everything in one command whenever you do get it running.

What you walk out with

- A running MySQL server, or a clear idea of where yours stuck.
- A four-table database you built by hand.
- Roughly fifteen queries you have seen run.
- Most of the mechanical work of HW2, already rehearsed.

Which MySQL?

This is the step most people get wrong, and the error does not show up until later. Oracle changed both the version scheme and the Windows installer recently, so a lot of what you will find online is stale.

What you will see offered on the download page.

Track	Version	For this course
LTS	9 . 7	Install this. Current long-term release.
LTS	8 . 4	Fine. Everything today works unchanged.
Innovation	26 . x	Avoid. Ships fast, breaks fast.
—	8 . 0	Do not. Support ended April 2026.

Two traps worth naming out loud

The tutorial trap. Almost every MySQL tutorial online was written against 8.0 and describes a bundled "MySQL Installer" that no longer drives the current install. If the screenshots do not match what you see, the tutorial is old, not you.

The version-number trap. MySQL now numbers its innovation track by calendar year, so 26.x is *newer* than 9.7 but less stable. Bigger number is not better here.

Nothing today is version-specific

Every statement in this deck is ordinary SQL. It runs on 8.4, on 9.7, and on MariaDB. Pick a version and stop thinking about it.

Windows

- 1 Download the **MySQL Community Server MSI** for Windows from dev.mysql.com/downloads/mysql/. Take the LTS release.
- 2 Run the MSI. It installs the server, then hands off to **MySQL Configurator**.
- 3 In Configurator, accept the defaults, with two exceptions below.
- 4 Set a **root password** you will actually remember. Write it down. There is no recovery path worth your Monday.
- 5 Leave **"Configure MySQL Server as a Windows Service"** checked and set to start automatically.
- 6 Let it apply the configuration. Watch for every step to go green.

The two settings that matter

Port 3306 — keep it unless something else already owns it. If Configurator complains, something else does.

Authentication method — take the default (strong password encryption). The "legacy" option exists only for old client software, and you have none.

Where this usually stalls

- **Missing Visual C++ Redistributable** — Configurator will say so and link it. Install, then continue.
- **Port 3306 in use** — an old MySQL, or XAMPP, is already running. Uninstall it or change the port.
- **A previous install's data directory** — the most common cause of a server that installs but will not start.

macOS and Linux

macOS

The DMG from the same download page works and is the path of least surprise. With Homebrew already installed:

```
$ brew install mysql
$ brew services start mysql
$ mysql_secure_installation
```

Homebrew installs with an empty root password; `mysql_secure_installation` is where you set one. Do not skip it.

Linux (Debian / Ubuntu)

```
$ sudo apt update
$ sudo apt install mysql-server
$ sudo systemctl start mysql
$ sudo mysql_secure_installation
```

Your distribution's packaged version may lag the LTS release. For this course that is fine.

On the lab machines

MySQL is already installed in CCP. You do not need to install anything to finish HW2 — the lab is a complete fallback.

If you install nothing today

You can still do every exercise on this deck. Read the companion script, predict each result, then check yourself once you have a server — in the lab, in office hours, or at home. Understanding the queries is the graded skill; installing a server is a one-time errand.

First contact

Before any SQL, prove the server is alive and that you can reach it. Thirty seconds here saves twenty minutes of confusion later.

Open a terminal and connect as root:

```
$ mysql -u root -p
```

It asks for the password you set. You should land here:

```
Welcome to the MySQL monitor.  
Your MySQL connection id is 8  
Server version: 9.7.x MySQL Community Server  
  
mysql>
```

Confirm you are really talking to a server:

```
SELECT VERSION();
```

"command not found: mysql"

The server installed, but its bin folder is not on your PATH. On Windows, either add C:\Program Files\MySQL\MySQL Server 9.7\bin to PATH, or use the **MySQL Command Line Client** from the Start menu, which sets it for you.

"Can't connect to local MySQL server"

The client is fine; the server is not running. On Windows, start the **MySQL** service in Services. On macOS, brew services start mysql. On Linux, sudo systemctl start mysql.

A GUI, if you want one

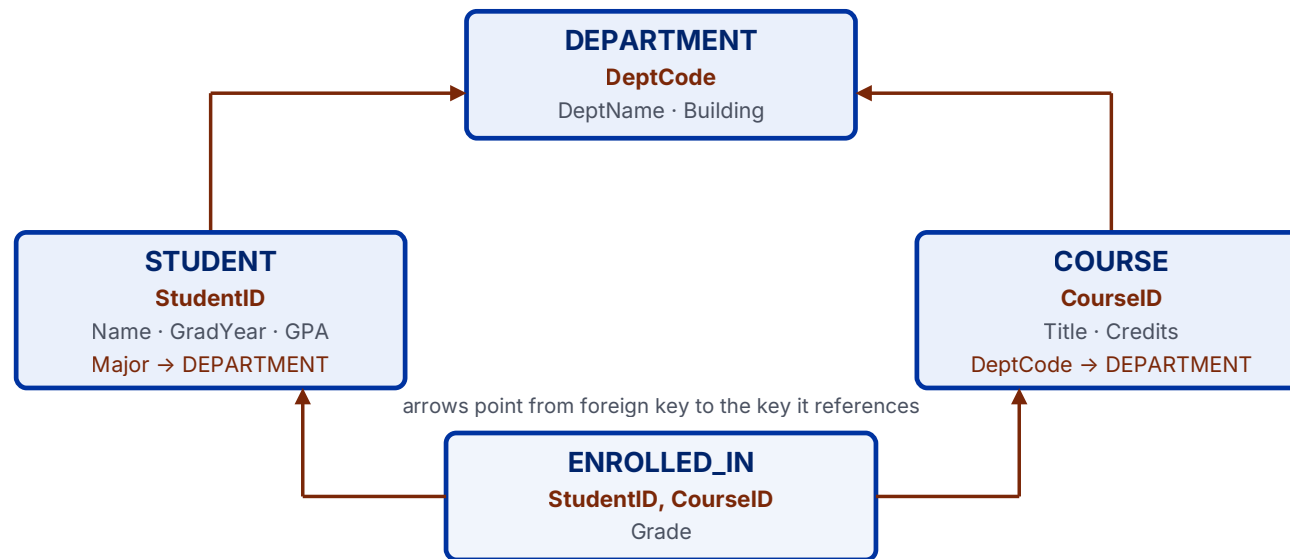
MySQL Workbench is the official graphical client and is worth having. We will stay in the terminal today, because the terminal is what shows you the errors plainly — and the errors are half of today's lesson.

The target

What we are building

The same UNIVERSITY mini-world from the ER lectures, now as actual tables.

Four relations, two of them pointing at a third.



Underlined attributes are primary keys. ENROLLED_IN is the M:N relationship from the Sep 16 lecture, carrying the pair as its key.

A database to put tables in

A fresh MySQL server has no database of yours on it. Make one, then step into it.

```
CREATE DATABASE university;  
USE university;
```

USE is not SQL doing anything to the data — it sets which database unqualified table names resolve against, for this connection only. Open a second terminal and you start over in no database at all.

```
-- where am I?  
SELECT DATABASE();  
SHOW DATABASES;
```

Vocabulary that trips people

MySQL uses **database** and **schema** as synonyms — CREATE SCHEMA is an alias for CREATE DATABASE. In the textbook, and in other systems like PostgreSQL, a schema is a namespace *inside* a database. Same word, two meanings; MySQL picked the looser one.

Running this a second time

CREATE DATABASE university fails if it already exists. To start clean, drop it first — and notice that MySQL will not ask you twice:

```
DROP DATABASE IF EXISTS university;
```

CREATE TABLE, line by line

```
CREATE TABLE DEPARTMENT (  
    DeptCode    CHAR(4)        NOT NULL,  
    DeptName    VARCHAR(40)    NOT NULL,  
    Building    VARCHAR(20),  
    PRIMARY KEY (DeptCode)  
);
```

```
CREATE TABLE STUDENT (  
    StudentID   CHAR(5)        NOT NULL,  
    Name        VARCHAR(40)    NOT NULL,  
    Major       CHAR(4),  
    GradYear    SMALLINT,  
    GPA         DECIMAL(3,2),  
    PRIMARY KEY (StudentID),  
    FOREIGN KEY (Major) REFERENCES DEPARTMENT(DeptCode)  
);
```

Three decisions in every column

1. **Type.** CHAR(4) is fixed width, VARCHAR(40) is up to 40. Codes are CHAR; names are VARCHAR.
2. **Nullability.** NOT NULL means the column must have a value. Here Major is nullable on purpose — an undeclared student has no major, and that is different from having a blank one.
3. **Constraints.** PRIMARY KEY and FOREIGN KEY are promises the server will enforce on every future write.

Why DEPARTMENT first

STUDENT references DEPARTMENT, so DEPARTMENT must already exist. Reverse the order and you get ERROR 1824, failed to open the referenced table. Create parents before children — the same rule will bite again when we insert rows.

The relationship table

```
CREATE TABLE COURSE (
  CourseID  VARCHAR(8)    NOT NULL,
  Title     VARCHAR(50)   NOT NULL,
  Credits   TINYINT       NOT NULL DEFAULT 3,
  DeptCode  CHAR(4)       NOT NULL,
  PRIMARY KEY (CourseID),
  FOREIGN KEY (DeptCode) REFERENCES DEPARTMENT(DeptCode)
);
```

```
CREATE TABLE ENROLLED_IN (
  StudentID CHAR(5)      NOT NULL,
  CourseID  VARCHAR(8)   NOT NULL,
  Grade     CHAR(2),
  PRIMARY KEY (StudentID, CourseID),
  FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),
  FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
);
```

This is the M:N mapping, in SQL

Three weeks ago we said a cardinality ratio survives into the relational schema as a **uniqueness constraint**, and nowhere else. Here is that claim as executable code.

PRIMARY KEY (StudentID, CourseID) — the *pair* is unique. Either column alone may repeat. That one line is the entire difference between M:N and the other three ratios.

DEFAULT earns its keep

Credits TINYINT NOT NULL DEFAULT 3 means an INSERT that omits Credits gets a 3 rather than an error. NOT NULL and DEFAULT together are how you say "this must have a value, and here is the value when you don't say."

Look at what you built

The server now holds a schema. Three commands read it back — and reading it back is how you catch a typo before it costs you an hour.

```
SHOW TABLES;
```

```
+-----+
| Tables_in_university |
+-----+
| COURSE                |
| DEPARTMENT            |
| ENROLLED_IN           |
| STUDENT               |
+-----+
```

```
SHOW CREATE TABLE ENROLLED_IN;
```

Prints the full definition the server actually stored, including everything it filled in for you. The most useful of the three, and the least known.

```
DESCRIBE STUDENT;
```

```
+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default |
+-----+-----+-----+-----+-----+
| StudentID  | char(5)       | NO   | PRI | NULL    |
| Name       | varchar(40)   | NO   |     | NULL    |
| Major      | char(4)       | YES  | MUL | NULL    |
| GradYear   | smallint      | YES  |     | NULL    |
| GPA        | decimal(3,2)  | YES  |     | NULL    |
+-----+-----+-----+-----+-----+
```

A sixth column, Extra, is omitted here — it is empty for every column of this table.

Read the Key column

PRI is the primary key. MUL means the column starts an index whose values may repeat — here, the foreign key on Major. Empty means unindexed. This column is the fastest way to confirm your constraints actually landed.

Putting data in

One INSERT can carry many rows. Naming the columns is optional and you should do it anyway — it survives someone adding a column later.

```
INSERT INTO DEPARTMENT (DeptCode, DeptName, Building) VALUES
('CS', 'Computer Science', 'CCP'),
('MATH', 'Mathematics', 'MB'),
('BIOL', 'Biology', 'SCI'),
('ENGL', 'English', 'LA');
```

```
INSERT INTO STUDENT (StudentID, Name, Major, GradYear, GPA)
VALUES
('S1001', 'Ana Reyes', 'CS', 2027, 3.72),
('S1003', 'Chloe Nguyen', 'MATH', 2027, 3.95),
-- ... six more ...
('S1008', 'Henry Patel', NULL, 2029, NULL);
```

Parents before children, again

DEPARTMENT, then STUDENT and COURSE, then ENROLLED_IN. Insert an enrollment before its student exists and the foreign key rejects it. The ordering rule you met in DDL is the same rule here, for the same reason.

Henry Patel is deliberate

A first-semester student: no declared major, no GPA yet. Both are NULL — which is not zero, not an empty string, and not a blank. It is the absence of a value.

He is in this data because he is going to embarrass a query in about ten minutes.

Now let's break it on purpose

A constraint you have never seen fire is a constraint you do not believe in. Four statements, four refusals — this is the whole argument for putting rules in the database instead of in application code.

```
-- 1. duplicate primary key
INSERT INTO STUDENT VALUES
('S1001', 'Imposter Reyes', 'CS', 2027, 3.00);
```

ERROR 1062 (23000)
Duplicate entry 'S1001' for key 'STUDENT.PRIMARY'

```
-- 2. foreign key with no parent
INSERT INTO STUDENT VALUES
('S9999', 'Test Student', 'ZZZZ', 2027, 3.00);
```

ERROR 1452 (23000)
Cannot add or update a child row: a foreign key constraint fails ... FOREIGN KEY (`Major`) REFERENCES `DEPARTMENT` (`DeptCode`)

```
-- 3. delete a parent that has children
DELETE FROM DEPARTMENT WHERE DeptCode = 'CS';
```

ERROR 1451 (23000)
Cannot delete or update a parent row: a foreign key constraint fails

```
-- 4. the composite key, proving itself
INSERT INTO ENROLLED_IN VALUES
('S1001', 'CS410', 'C');
```

ERROR 1062 (23000)
Duplicate entry 'S1001-CS410' for key 'ENROLLED_IN.PRIMARY'

Look closely at the last one

The server printed the *pair*, hyphenated. Ana may take many courses and CS410 may hold many students — but not this pair twice. That error message is the M:N constraint telling you its own name.

SELECT · FROM · WHERE

Three clauses, and they do not execute in the order you read them.

```
SELECT Name, GPA      -- 3. which columns
FROM STUDENT          -- 1. which table
WHERE GPA > 3.5;      -- 2. which rows
```

Read it as: start with a table, throw away the rows that fail the condition, then keep only the columns asked for.

In the algebra from Unit 3 this is $\pi_{\text{Name,GPA}}(\sigma_{\text{GPA}>3.5}(\text{STUDENT}))$ — SELECT is the projection π , and WHERE is the selection σ . The names collide unhelpfully; the operations do not.

STUDENT — the eight rows we will query all period.

StudentID	Name	Major	GradYear	GPA
S1001	Ana Reyes	CS	2027	3.72
S1002	Ben Okafor	CS	2026	3.15
S1003	Chloe Nguyen	MATH	2027	3.95
S1004	Diego Santos	CS	2028	2.88
S1005	Emma Larsen	BIOL	2026	3.40
S1006	Farid Haddad	MATH	2028	3.61
S1007	Grace Kim	CS	2027	3.33
S1008	Henry Patel	NULL	2029	NULL

Change the WHERE. Watch the rows go.

Same table, same SELECT, same FROM. Only the condition moves. Pick one and see which of the eight rows survive it.

```
SELECT * FROM STUDENT;
```

No WHERE clause at all: every row of the table is returned. All 8 students match.

Rows marked **dropped** fail the condition and do not appear in the result.

Result	StudentID	Name	Major	GradYear	GPA
kept	S1001	Ana Reyes	CS	2027	3.72
kept	S1002	Ben Okafor	CS	2026	3.15
kept	S1003	Chloe Nguyen	MATH	2027	3.95
kept	S1004	Diego Santos	CS	2028	2.88
kept	S1005	Emma Larsen	BIOL	2026	3.40
kept	S1006	Farid Haddad	MATH	2028	3.61
kept	S1007	Grace Kim	CS	2027	3.33
kept	S1008	Henry Patel	NULL	2029	NULL

The ladder

Call out the answer before I press enter. Row counts are from the data we just loaded.

```
-- 1. everything → 8
SELECT * FROM STUDENT;

-- 2. just two columns → 8
SELECT Name, Major FROM STUDENT;

-- 3. one condition → 4
SELECT Name FROM STUDENT
WHERE Major = 'CS';

-- 4. a comparison → 3
SELECT Name, GPA FROM STUDENT
WHERE GPA > 3.5;

-- 5. two conditions → 2
SELECT Name FROM STUDENT
WHERE Major = 'CS' AND GradYear = 2027;

-- 6. a set of values → 3
SELECT Name, Major FROM STUDENT
WHERE Major IN ('MATH', 'BIOL');
```

```
-- 7. a range, endpoints included → 3
SELECT Name, GPA FROM STUDENT
WHERE GPA BETWEEN 3.0 AND 3.5;

-- 8. pattern match; % is any text → 1
SELECT Name FROM STUDENT
WHERE Name LIKE 'A%';

-- 9. sort it → 8
SELECT Name, GPA FROM STUDENT
ORDER BY GPA DESC;

-- 10. drop duplicates → 4
SELECT DISTINCT Major FROM STUDENT;
```

Two things to notice when these run

Exercise 8. LIKE 'A%' and LIKE 'a%' return the same row. MySQL's default collation ignores case. That is a MySQL choice, not an SQL one.

Exercise 9. Henry sorts last. NULL is not low, not high, not zero — the sort just has to put it somewhere, and MySQL puts it last when descending.

Six plus one is not eight

Here is the promised embarrassment. Two queries that look like exact opposites, run against eight students.

```
SELECT Name FROM STUDENT WHERE GPA > 3.0;
```

6 rows: Ana, Ben, Chloe, Emma, Farid, Grace

```
SELECT Name FROM STUDENT WHERE GPA <= 3.0;
```

1 row: Diego

6 + 1 = 7. There are eight students. Where is Henry?

Three-valued logic

Henry's GPA is NULL — unknown. NULL > 3.0 is not true. It is not false either. It evaluates to **UNKNOWN**, and WHERE keeps a row only when the condition is *true*. Unknown is discarded, both times.

So Henry falls out of a query and out of its complement. Nothing is broken; this is the defined behavior, and it is the single most common source of silently wrong answers in SQL.

-- this returns nothing at all, ever

```
SELECT Name FROM STUDENT WHERE GPA = NULL;
```

-- this is how you ask

```
SELECT Name FROM STUDENT WHERE GPA IS NULL;
```

The habit to build

Every time you write a WHERE clause on a nullable column, ask what it does to the NULLs. DESCRIBE tells you which columns those are — it is the Null: YES column.

Two tables at once

Put two tables in FROM and you get every pairing of their rows — the cross product from Unit 3. Eight students times fifteen enrollments:

```
SELECT COUNT(*) FROM STUDENT, ENROLLED_IN;
```

120

Nonsense rows, pairing every student with every enrollment including other people's. The WHERE clause is what makes it meaningful:

```
SELECT COUNT(*) FROM STUDENT, ENROLLED_IN
WHERE STUDENT.StudentID = ENROLLED_IN.StudentID;
```

15

A join is a cross product plus a condition. That is not a metaphor — it is what the engine is doing.

With aliases, asking a real question:

```
SELECT S.Name, E.Grade
FROM STUDENT S, ENROLLED_IN E
WHERE S.StudentID = E.StudentID
AND E.CourseID = 'CS410';
```

+	-----	+	-----	+
	Name		Grade	
+	-----	+	-----	+
	Ana Reyes		A	
	Ben Okafor		B	
	Chloe Nguyen		A-	
	Diego Santos		NULL	
	Grace Kim		A-	
+	-----	+	-----	+

Forget the join condition and you get 120 rows

With n tables in FROM you need $n-1$ join conditions. Miss one and the result explodes rather than erroring — it is a perfectly legal query returning garbage. Count your conditions before you trust the output.

Wrap-up

Closing notes

What today established

- **DDL writes the rules; DML moves the data.** CREATE defines what may exist; INSERT, UPDATE and DELETE work inside those limits.
- **Constraints are the database's job.** You saw four bad writes refused with no application code involved.
- **Order follows the references.** Parents before children, in DDL and in DML both.
- **WHERE is selection σ ; SELECT is projection π .** The algebra from Unit 3, with worse names.
- **NULL is not a value.** It breaks complements, and $=$ NULL never matches anything.
- **A join is a cross product plus a condition.**

HW2 is due Wednesday

Its setup section is what we just did. If your install worked, you are through the part that takes unpredictable time. If it did not, use the lab machines or come to office hours tomorrow — do not spend Tuesday night fighting an installer.

Before Wednesday

1. Get a server running somewhere you can reach.
2. Run the companion script end to end.
3. Work the ladder on slide 16 without looking at the counts.
4. Write one query the deck does not contain, and see if it does what you expected.

Next

Grouping and aggregation — GROUP BY and HAVING — then views, assertions and triggers. The last query on the resources slide is a preview.

Reference

Resources

Downloads

- **MySQL Community Server** — dev.mysql.com/downloads/mysql/
Take an LTS release (9.7 or 8.4), not the innovation track.
- **MySQL Workbench** — dev.mysql.com/downloads/workbench/
Optional graphical client. The current generation is a rewrite; expect it to look unlike older screenshots.

Documentation

- **MySQL Reference Manual** — dev.mysql.com/doc/refman/9.7/en/
Chapter 2 is installation; Chapter 3, "Tutorial," covers today's material at more length.

Course materials

- `university_bootstrap.sql` — rebuilds everything from today in one command.
On Canvas with these slides.
- Elmasri & Navathe, **Ch. 6** — SQL data definition and basic retrieval. Ch. 7 for what comes Wednesday.

Running the companion script

```
mysql> SOURCE university_bootstrap.sql
```

From a shell instead: `mysql -u root -p < university_bootstrap.sql`

Getting unstuck

- **Office hours** — bring the laptop and the exact error text, not a description of it.
- **The lab machines in CCP** — MySQL is installed; nothing in HW2 requires your own machine.
- **Error numbers are searchable.** "MySQL 1452" beats a sentence describing the problem.

A preview of Wednesday

```
SELECT  C.Title, COUNT(*) AS Enrollment
FROM    COURSE C, ENROLLED_IN E
WHERE   C.CourseID = E.CourseID
GROUP BY C.Title;
```

Run it. Databases comes back with 5. Work out why before Wednesday.